Manuscript submitted for BCSE307P Compiler Design, Vellore Institute of Technology, October 2026.

Digital Object Identifier

NEMO-C: Teaching a Compiler When *Not* to Offload to Processing-in-Memory

SOMESH TRIPATHI[1], SMEEHEN[1], SWATI SAUMYA[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India (Reg. Nos. 24BCE0533, 24BCE0502, 24BCE0253)

Corresponding author: Somesh Tripathi (e-mail: somesh.tripathi2024@vitstudent.ac.in).

This work received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

ABSTRACT Processing-in-memory (PIM) places small processors inside DRAM so that data-intensive code can run where its data already lives, and commercial PIM devices are now available. Their programmers, however, must decide by hand which loops to move to the memory-side processors, and a wrong choice can make a program slower than running it on the CPU alone. Current PIM software stacks either leave this choice to the programmer through libraries and data-parallel interfaces, or make it inside domain-specific compilers for neural networks and in-DRAM bit-serial operations. None of them gives a general loop compiler a static, explainable rule that refuses an offload when the cost of moving data, launching the device and losing cache reuse outweighs the gain. We present NEMO-C, a compiler built from scratch in C++17 with Flex and Bison, that decides for every loop nest whether it should run on the host or on 64 DPUs. Its rule combines arithmetic intensity, data reuse, dependence-based parallelism, partitionability and a launch-cost size check, and loop fission, interchange and fusion turn some rejected loops into profitable ones. On 15 benchmarks in a cycle-level simulator, NEMO-C is never slower than the host, reaches a geometric-mean speedup of 1.42× (up to 7.47×), and avoids the 0.63×, 0.70× and 0.43× slowdowns that blind offloading causes; all 45 runs are bit-identical. Future work will calibrate the cost model on real UPMEM hardware and extend it to irregular and multi-kernel programs.

INDEX TERMS Compiler optimization, cost model, data movement, dependence analysis, near-memory computing, processing-in-memory

I. INTRODUCTION

FOR many data-intensive programs the time spent moving data between main memory and the processor now outweighs the time spent computing on it. Mutlu et al. argue that this movement is the dominant cost for a broad class of workloads and that computation should be performed where the data reside [1]. Processing-in-memory (PIM) and the closely related near-memory computing (NMC) attach processing elements to, or place them inside, the memory modules so that streaming code does not have to cross the off-chip bus at all [2], [3]. The idea is no longer only an academic one: UPMEM ships DDR4 modules in which every DRAM chip contains general-purpose in-order cores called DPUs [4], and Samsung has built a function-in-memory HBM2 device with programmable compute units next to the banks [5].

Placing a processor next to memory does not by itself make a program faster. A loop sent to the DPUs has to pay for launching and synchronising the device, for copying its operands into the DPUs' small scratchpads, and for giving up the host's caches, which may already have been serving most of its accesses [6]. A loop that does a lot of arithmetic per byte, that re-reads its data, or whose iterations need data owned by other DPUs, can therefore run more slowly in memory than on the CPU. Studies of consumer-device and general-purpose workloads confirm that the benefit depends strongly on the kind of bottleneck a function exhibits [7], [8]. Today the programmer of a PIM system has to make this call for every kernel, by hand. The problem we address is to make the call automatically, at compile time, and to make it correctly in both directions: offload the loops that gain, and refuse the ones that lose.

Two families of existing work bear on this problem. The first characterises real PIM hardware and gives programmers practical guidance. Benchmark studies of the UPMEM system, and evaluations of machine-learning and transcendental-function workloads on it, show which kinds of kernels suit DPUs and which do not [9], [10]. Industry prototypes report

large gains for the memory-bound parts of recommendation and neural-network inference when those parts are chosen carefully [5], [11].

The second family builds programming support around PIM: libraries, data-parallel frameworks and domain-specific compilers that generate PIM code from a higher-level description. These tools reduce the effort of writing PIM code considerably. Classical compiler technology supplies the remaining ingredients, namely dependence analysis, loop transformations, dataflow analysis and register allocation, all of which are mature and well documented [12]. Section II reviews fifteen recent systems of both kinds in detail [13]–[27].

The shortfall of the first family is that its knowledge stays in papers and in programmers' heads. A characterisation study can say that kernels with heavy inter-DPU communication scale poorly, but nothing in the tool chain applies that observation to a new loop. The programmer still decides, still has to estimate data movement by hand, and receives no warning when an offload is unprofitable.

The shortfall of the second family is that most of its tools assume the decision has already been made. A library or a data-parallel framework generates efficient PIM code for whatever the programmer marks for PIM; a neural-network compiler maps layers that are known in advance to suit the device. Few of them combine a static cost model with dependence analysis for general loop nests, few of them use loop transformations to make a rejected loop profitable, and few explain their decision in terms a programmer can check. Among the tools we reviewed, only the framework of Devic et al. treats refusing an offload as a first-class outcome, and none pairs that refusal with loop transformations that make the loop profitable instead.

This paper proposes NEMO-C (a NEar-Memory-aware Optimising Compiler), which closes this gap for affine loop programs. For every loop nest, NEMO-C computes five classical properties: arithmetic intensity, self-temporal reuse weighted by its share of the traffic, parallelism from GCD and Banerjee dependence tests, partitionability under the owner-computes rule, and an estimate of whether the nest is large enough to pay for launching the DPUs. It offloads the nest only if all checks pass. When a nest fails, loop fission, interchange and fusion are tried, and the nest is decided again. Offloaded nests are partitioned across 64 DPUs, flush and invalidate operations are placed by a forward dataflow analysis, and DPU assembly is generated with scratchpad tiling and Chaitin–Briggs graph-colouring register allocation.

The approach suits the problem because each check corresponds to a concrete reason why offloading can fail, and each is computed by an analysis whose correctness is well understood. The decision is static, so it needs neither hardware nor profiling runs, and it is explainable: the compiler prints every number and every check behind a verdict. The advantage shows in the results. Across 15 benchmarks NEMO-C is never slower than the host, while blind offloading loses on four of them, and a single-metric rule based on arithmetic intensity loses on three. Every configuration of every benchmark produces bit-identical results, including runs that execute the generated DPU assembly itself.

The rest of the paper is organised as follows. Section II reviews fifteen related systems and the gap each leaves. Section III formulates the problem and presents the architecture, modules, equations and algorithms of NEMO-C. Section IV describes the implementation, the experimental setup and the results for six metrics, and compares NEMO-C with three related works. Section V concludes and outlines future work.

II. RELATED WORKS

This section reviews fifteen recent works on PIM characterisation, programming support, compilation and hardware. For each we summarise the solution, the methodology and tools, the reported results, and the research gap relative to automatic offload decisions for general loop programs. Table 1 summarises them.

A. EXPERIMENTAL CHARACTERISATION OF A REAL PIM SYSTEM (PRIM)

Solution. Gómez-Luna et al. present the first comprehensive experimental analysis of the publicly available UPMEM PIM architecture, in which DRAM arrays and in-order DPU cores share the same chip [13]. Their aim is to tell programmers and architects what the hardware can and cannot do well.

Methodology and tools. The study first measures the limits of the hardware with microbenchmarks that isolate the DPUs' compute throughput and the bandwidth of their local memory and of host–DPU transfers. These measurements expose, for example, how arithmetic throughput depends on the operation and data type, and how the scratchpad and the DRAM bank behave under different access patterns.

The authors then introduce PrIM, a suite of 16 memory-bound workloads drawn from linear algebra, databases, graph processing, neural networks and other domains, and port each one by hand to UPMEM. The workloads were run on two UPMEM systems with 640 and 2,556 DPUs, and their scaling and performance were compared with conventional processors.

Results. The study finds that workloads with streaming access, little or no inter-DPU communication and simple arithmetic benefit most, while workloads that need frequent communication between DPUs, which on UPMEM must pass through the host, scale poorly.

Research gap. The guidance is empirical and is applied by hand; no compiler applies it automatically to a new loop.

B. TO PIM OR NOT FOR GENERAL-PURPOSE PROCESSING IN DDR SYSTEMS

Solution. Devic et al. study PIM hardware that places a general-purpose RISC-V core at each memory bank, and present a software compilation framework that decides what, whether, when and how to offload work to these bank-level engines [14]. They also propose a vector-engine extension to the bank cores.

Methodology and tools. The framework starts from off-the-shelf C/C++ applications and identifies regions that could execute on the bank-level cores. It then decides, region by region, whether offloading is worthwhile, which makes the work one of the few to treat offloading as a decision rather than a default.

The evaluation is simulation-based; the authors' public repository (BLIMP, Bank-Level In-memory Processing) provides the benchmarks, the instrumentation, the compilation flow and the simulation setup. They also study how the number of memory banks and the relative speed of host and PIM cores change the answer.

Results. The authors report that PIM is not always beneficial for off-the-shelf applications, so selective offloading matters, and that bank count and core speeds shift the to-PIM-or-not decision.

Research gap. The work targets bank-level RISC-V cores and whole application regions; loop-level enabling transformations and a per-loop explanation of each verdict are outside its stated scope.

C. CINM: A COMPILATION INFRASTRUCTURE FOR IN- AND NEAR-MEMORY COMPUTING

Solution. Khan et al. present Cinnamon (CINM), an end-to-end compilation flow that targets both compute-in-memory (CIM) devices, such as memristive crossbars, and compute-near-memory (CNM) devices, such as UPMEM, from one program representation [15]. They argue that existing programming models are too low-level and too tied to a particular system.

Methodology and tools. CINM uses hierarchical abstractions: a program is lowered progressively from a device-independent representation to device-specific ones, and optimisations are applied at each level of the lowering pipeline. This lets common optimisations be shared across devices while device-specific ones are applied late.

The evaluation compiles a set of benchmarks for a real CNM system (UPMEM) and for memristor-based CIM accelerators, and compares the generated code with state-of-the-art hand-written implementations.

Results. The authors report that the generated code is comparable to, or better than, the state-of-the-art implementations.

Research gap. CINM generates code for kernels that are mapped to the in-memory device; it does not decide, from a cost model, that a kernel should stay on the host.

D. SIMPLEPIM: A SOFTWARE FRAMEWORK FOR PRODUCTIVE PIM PROGRAMMING

Solution. Chen et al. propose SimplePIM, a high-level framework that lets the host drive PIM devices through iterator functions over arrays (such as map and reduce) and supplies communication primitives between the host and the PIM cores [16].

Methodology and tools. The programmer expresses a computation as array iterators with user-defined element functions; SimplePIM handles the distribution of data, the launching of the DPUs and the gathering of results. Its interface handles arrays of arbitrary element types.

The framework is implemented on the UPMEM system and evaluated on six applications, including vector addition, reduction, histogram, K-means clustering and linear and logistic regression, against hand-optimised UPMEM baselines.

Results. SimplePIM reduces lines of code by 66.5% to 83.1% and achieves speedups of 10% to 37% over hand-optimised code in three applications, with comparable performance in the other three.

Research gap. The programmer still chooses what runs on PIM; the framework does not reject an unprofitable offload.

E. DAPPA: A DATA-PARALLEL PROGRAMMING FRAMEWORK FOR PIM

Solution. Oliveira et al. propose DaPPA, a framework that hides UPMEM's explicit data management and workload partitioning behind data-parallel patterns [17].

Methodology and tools. DaPPA has three parts: a set of five data-parallel pattern primitives, a dataflow programming interface that lets patterns be composed, and a dynamic template-based compiler that turns the description into an optimised UPMEM binary. Memory allocation, data distribution and gathering, and parallelisation are handled automatically.

It is evaluated on six workloads from the PrIM suite on a real UPMEM system and compared with the hand-tuned PrIM implementations in performance and in programming effort.

Results. DaPPA improves end-to-end performance by 2.1× on average over the hand-tuned versions and reduces lines of code by 94% on average.

Research gap. Like SimplePIM, it optimises code that the programmer has already decided to run on PIM.

F. PIMFLOW: COMPILER AND RUNTIME SUPPORT FOR CNNS ON DRAM-PIM

Solution. Shin et al. present PIMFlow, an end-to-end compiler and runtime that runs convolutional neural networks on a GPU whose memory contains DRAM-PIM units [18]. They observe that recent CNNs increasingly use computationally lightweight layers, which suit PIM.

Methodology and tools. PIMFlow transforms the model graph to create parallelism between nodes so that some layers can run on the GPU and others on PIM at the same time, and it explores task-parallel and data-parallel execution scenarios to minimise execution time.

It provides a code-generating back end and an execution engine for DRAM-PIM, and it is integrated with the TVM compiler for the graph-level optimisations.

Results. PIMFlow achieves up to 82% end-to-end speedup and an average 26% energy reduction for CNN inference.

Research gap. The decision is made for neural-network layers inside a GPU system; general affine loop nests on a host CPU are not addressed.

G. DAMOV: CLASSIFYING DATA-MOVEMENT BOTTLENECKS

Solution. Oliveira et al. present DAMOV, a methodology and benchmark suite for finding the sources of data-movement bottlenecks and choosing the right mitigation for them, with near-data processing as the main case study [19].

Methodology and tools. The methodology has three steps and uses four key metrics to classify each function into one of six classes of memory bottleneck. It was applied to 77K functions from 345 applications.

From this characterisation the authors select 144 functions as the open-source DAMOV suite and use the classification to decide which mitigation mechanism, such as near-data processing, suits each class.

Results. The study shows that the right mitigation depends on the bottleneck class, and it releases the suite and tool chain for further studies.

Research gap. The classification is an offline profiling study; it is not embedded in a compiler that acts on it per loop.

H. SPARSEP: SPARSE MATRIX–VECTOR MULTIPLICATION ON REAL PIM

Solution. Giannoula et al. present SparseP, the first library of sparse matrix–vector multiplication (SpMV) kernels for real PIM systems, together with the first analysis of SpMV on such hardware [20].

Methodology and tools. SparseP provides 25 SpMV kernels across four compressed formats (CSR, COO, BCSR and BCOO) and several schemes for partitioning the matrix and balancing load across DPUs.

The kernels are evaluated on a real UPMEM system with 2,528 PIM cores and compared with CPU and GPU systems in performance and energy.

Results. The study identifies how partitioning and load balancing decide whether this irregular, memory-bound kernel benefits from PIM.

Research gap. The library hand-optimises one kernel; it does not provide a general rule for other loops.

I. CONDA: CACHE COHERENCE FOR NEAR-DATA ACCELERATORS

Solution. Boroumand et al. propose CoNDA, a coherence mechanism for near-data accelerators (NDAs) that sit off-chip near main memory [21]. They find that existing coherence mechanisms erase most of the benefit of NDAs.

Methodology and tools. CoNDA lets the NDA execute a kernel optimistically, as if it already held the needed coherence permissions. It records the memory accesses of the NDA and of the rest of the system during the kernel and resolves conflicts afterwards, skipping coherence requests that turn out to be unnecessary.

It is evaluated on graph-processing and hybrid in-memory database workloads and compared with prior coherence mechanisms and with an ideal, zero-cost mechanism.

Results. CoNDA improves performance by 19.6% over the best prior mechanism on modest data sets (38.3% on large ones) and comes within 10.4% of the ideal mechanism.

Research gap. CoNDA is a hardware mechanism; a compiler that knows which arrays a kernel reads and writes can place the needed flushes and invalidations statically.

J. INFINITY STREAM: IN-/NEAR-MEMORY FUSION

Solution. Wang et al. present Infinity Stream, which combines in-memory computing in large last-level caches with near-memory computing in a portable and programmer-friendly way [22].

Methodology and tools. The work introduces a tensor dataflow graph (tDFG) as an intermediate representation that captures both in-memory and near-memory computation. A two-phase, JIT-based compilation lowers the tDFG to in-memory commands, which keeps programs portable across microarchitectures.

It addresses automatic parallelisation, data transposition for bit-serial logic and the combination of in- and near-memory computing, and it is evaluated with a cycle-accurate simulator on fp32 data-processing workloads.

Results. Infinity Stream reports a 2.6× speedup, a 75% traffic reduction and 2.4× energy efficiency over a state-of-the-art near-memory technique.

Research gap. It targets cache-resident bit-serial computation, not near-bank DPUs, and its scope is the mapping of computation rather than a refusal rule.

K. MIMDRAM: MIMD PROCESSING-USING-DRAM

Solution. Oliveira et al. present MIMDRAM, an end-to-end processing-using-DRAM (PUD) system that exploits fine-grained DRAM to run multiple instruction streams in each DRAM subarray [23].

Methodology and tools. Because smaller segments of a wide DRAM row can be activated independently, MIMDRAM executes MIMD operations across segments, with SIMD inside each segment, which uses the DRAM array more efficiently than whole-row SIMD.

Its compiler passes take unmodified C/C++ code, identify loops suitable for PUD execution, transform them to use PUD operations, and schedule independent operations to run concurrently.

Results. MIMDRAM reports 34× the performance, 14.3× the energy efficiency, 1.7× the throughput and 1.3× the fairness of a state-of-the-art PUD framework, with an area overhead of 1.11% of a DRAM chip and 0.6% of a CPU die.

Research gap. Its loop selection is for bit-serial in-DRAM execution, not for near-bank cores that pay launch and DMA costs.

L. SIMDRAM: BIT-SERIAL SIMD PROCESSING USING DRAM

Solution. Hajinazar et al. present SIMDRAM, a flexible, general-purpose processing-using-DRAM framework that

uses each DRAM column as an independent SIMD lane for bit-serial operations [24].

Methodology and tools. SIMDRAM works in three steps: it builds an efficient representation of a desired operation using majority (MAJ) and NOT logic, maps the operands and the operation onto DRAM rows and commands, and executes the operation.

The framework is evaluated against a state-of-the-art in-DRAM computing mechanism and against a CPU and a high-end GPU.

Results. SIMDRAM provides up to 5.1× higher operation throughput and 2.5× higher energy efficiency than the prior in-DRAM mechanism, with less than 1% DRAM chip area overhead, and is 257× and 31× more energy-efficient than the CPU and GPU.

Research gap. It provides operations for in-DRAM execution but leaves the choice between host and memory to the programmer.

M. HBM-PIM: ARCHITECTURE AND SOFTWARE STACK ON COMMERCIAL DRAM

Solution. Lee et al. present a PIM architecture built on commercial HBM2 DRAM, with a lightweight PIM unit at the I/O boundary between the DRAM cells and the peripheral circuitry, shared by each pair of banks [25].

Methodology and tools. The device is implemented in a 20 nm DRAM technology and integrated with an unmodified commercial processor; host column read and write commands are reused to trigger PIM execution, which keeps it compatible with JEDEC memory controllers.

The authors developed the software stack needed to run existing applications without changing their source code and evaluated it on memory-bound neural-network kernels and full applications.

Results. PIM improves memory-bound neural-network kernels by 11.2× and applications by 3.5×, and reduces energy per bit transferred by 3.5×.

Research gap. The stack offloads operations known to be memory-bound in machine learning; general loop nests and the decision to refuse offloading are not addressed.

N. PATHFINDING FUTURE PIM ARCHITECTURES (UPIMULATOR)

Solution. Hyun et al. demystify UPMEM's commercial PIM technology and use the insight to identify features that future PIM architectures will need [26].

Methodology and tools. They built uPIMulator, which compiles UPMEM-PIM source code into machine-level instructions and feeds them to a cycle-level performance simulator.

With the simulator they characterise the UPMEM design in detail and run case studies on possible architectural changes.

Results. The work provides a validated simulation infrastructure and a set of architectural directions for general-purpose PIM.

Research gap. It is an architecture-exploration tool; deciding which code to offload is left to the programmer.

O. PIMCOMP: A COMPILATION FRAMEWORK FOR CROSSBAR-BASED PIM

Solution. Sun et al. present PIMCOMP, a universal compilation framework for non-volatile-memory crossbar-based PIM accelerators for deep neural networks [27].

Methodology and tools. PIMCOMP is built on an abstract accelerator model that matches the common crossbar, in-memory-accelerator, tile and chip hierarchy. Its compiler runs four stages: node partitioning, weight replication, core mapping and dataflow scheduling.

It offers two modes with different inter-layer pipeline granularities, one for high throughput and one for low latency, and is compared with the PUMA compiler.

Results. PIMCOMP reports 1.6× higher throughput and 2.4× lower latency than PUMA.

Research gap. It targets DNN layers on crossbars; general loop nests and host-versus-PIM profitability are outside its scope.

III. PROPOSED SYSTEM

A. PROBLEM FORMULATION

A program is a sequence of host blocks and kernels over multidimensional arrays; every kernel is a list of loop nests $\mathcal{L} = \{L_1, \ldots, L_m\}$ whose loop bounds are compile-time constants. The target machine has a host CPU with a cache and $P = 64$ DPUs, each with a private DRAM bank and a 64 KB scratchpad (WRAM). DPUs cannot exchange data directly; anything one DPU needs from another must pass through the host. The compiler must choose a mapping $M : \mathcal{L} \to \{\text{host}, \text{dpu}\}$ and generate code for it. The mapping must not change the program's results, and it should minimise total execution time.

We ask for more than a low average time. A mapping is acceptable only if no nest is offloaded when offloading makes it slower; formally, for every nest, $T(L, M(L)) \leq T(L, \text{host})$. The decision must be made at compile time from static analysis alone, without hardware, profiling runs or input data. The cost of a nest on the DPUs is not known exactly at compile time, so the compiler must reason with conservative estimates and must be able to explain each verdict. When a nest is rejected, the compiler may apply semantics-preserving loop transformations and decide again.

B. SYSTEM ARCHITECTURE

Fig. 1 shows the architecture of NEMO-C. It is organised as nine phases, labelled A to I, which follow the classical front end, middle end and back end of a compiler and add a simulator for validation. The front end (A) scans the `.nmc` source with a Flex-generated DFA, parses it with a Bison-generated LALR(1) parser that has no grammar conflicts, and checks it semantically against a scoped symbol table. The middle end lowers the tree to three-address code that keeps loop nests intact and builds a control-flow graph on demand

TABLE 1. Summary of related work and the gap each leaves for automatic offload decisions on general loop programs.

Work	Approach	Platform	Gap relative to NEMO-C
PrIM [13]	Microbenchmarks and 16 hand-ported workloads	Real UPMEM (640, 2,556 DPUs)	Guidance applied by hand, no automatic decision
Devic *et al.* [14]	Compilation framework deciding what and whether to offload	Simulated RISC-V core per bank	Region level; no loop transformations or per-loop explanation reported
CINM [15]	Hierarchical progressive lowering	UPMEM and memristive CIM	Compiles mapped kernels; no refusal rule
SimplePIM [16]	Host-driven array iterators	Real UPMEM	Programmer chooses offloads
DaPPA [17]	Data-parallel patterns, template compiler	Real UPMEM	Programmer chooses offloads
PIMFlow [18]	CNN graph transforms, TVM integration	GPU with DRAM-PIM	CNN layers only
DAMOV [19]	Bottleneck classification of 77K functions	Profiling and simulation	Offline study, not in a compiler
SparseP [20]	25 SpMV kernels, 4 formats	Real UPMEM (2,528 cores)	One hand-optimised kernel
CoNDA [21]	Optimistic hardware coherence	Simulated NDA	Hardware; no static placement
Infinity Stream [22]	tDFG IR, JIT lowering	Simulated LLC in-memory	Cache bit-serial; no refusal rule
MIMDRAM [23]	Fine-grained MIMD PUD, compiler passes	Simulated DRAM	Bit-serial PUD, not near-bank cores
SIMDRAM [24]	MAJ/NOT bit-serial framework	Simulated DRAM	Programmer chooses offloads
HBM-PIM [25]	PIM units at bank I/O, software stack	Real HBM2 prototype	ML operations only
uPIMulator [26]	Compiler and cycle-level simulator	Simulated UPMEM	Architecture exploration only
PIMCOMP [27]	Four-stage DNN mapping	Crossbar NVM PIM	DNN-specific
NEMO-C (this work)	**Six-check static cost model with dependence analysis and enabling transformations**	**Cycle-level UPMEM-style model**	**Decides per loop nest, refuses unprofitable offloads, explains each verdict**

(B), runs five classical optimisations (C) and computes access matrices, dependences and parallel loops (D).

The decision phase (E) is the centre of the design. It receives the dependence information and the IR of each nest and either accepts the nest for the DPUs or rejects it; a rejected nest is passed to the transformation phase (F), and the result is decided again. Accepted nests are partitioned across the DPUs and given coherence operations (G), after which the back end (H) generates DPU assembly with scratchpad tiling and graph-colouring register allocation, together with host C code that moves data and launches the DPUs. Rejected nests remain ordinary host code. Finally the simulator (I) runs the program three ways, checks that all results are bit-identical, executes the generated DPU assembly and reports cycles, memory-bus bytes and energy.

C. MODULES AND THEIR FUNCTIONS

Table 2 lists the modules. The compiler consists of 43 source files and about 11,400 lines of C++17, Flex and Bison, with one file per pass so that each analysis can be tested and explained in isolation; each file begins with a comment naming the compiler-theory concept and the standard algorithm it implements. A separate test suite of eight programs with 365 checks exercises every pass, and a browser-based workbench (`web/server.py`) runs the same compiler on any program for demonstration.

TABLE 2. Modules of NEMO-C and their functions.

Ph.	Module (files)	Function
A	lexer.l	DFA scanner, maximal munch, line and column tracking
A	parser.y, ast.cpp	LALR(1) parser, 0 conflicts, panic-mode recovery, AST
A	symtab.cpp, semantic.cpp	Scoped symbol table; type, bound and loop-index checks
B	lower.cpp, ir.h, cfg.cpp	Three-address IR with nested loops; CFG on demand
C	opt_constfold, opt_cse, opt_dce, opt_indvar, opt_strength	Constant propagation, CSE, dead-code elimination, induction variables, strength reduction
D	pass_affine, pass_dependence	Access matrices; GCD and Banerjee tests; direction vectors; reductions
E	pass_profitability	Six-check offload decision and plain-English trace
F	pass_fission, pass_interchange, pass_fusion	Allen–Kennedy fission, loop interchange, loop fusion
G	pass_partition, pass_coherence	Owner-computes partitioning; flush/invalidate placement
H	codegen_dpu, regalloc, codegen_host	PIM assembly with tiling; Chaitin–Briggs allocation; host C
I	simulator, dpu_interp	Cycle, byte and energy model; execution of the generated DPU code

D. FLOW DIAGRAM

Fig. 2 shows how one program travels through the compiler. After parsing and checking, any error is reported with its file, line and column, and the parser recovers at the next statement so that all errors are reported in one run. A correct program is lowered and optimised, and then every loop nest is analysed and decided on its own. A nest that passes is partitioned and compiled for the DPUs. A nest that fails is offered to the transformations; if one applies, the transformed nests are decided again, otherwise the nest stays on the host.

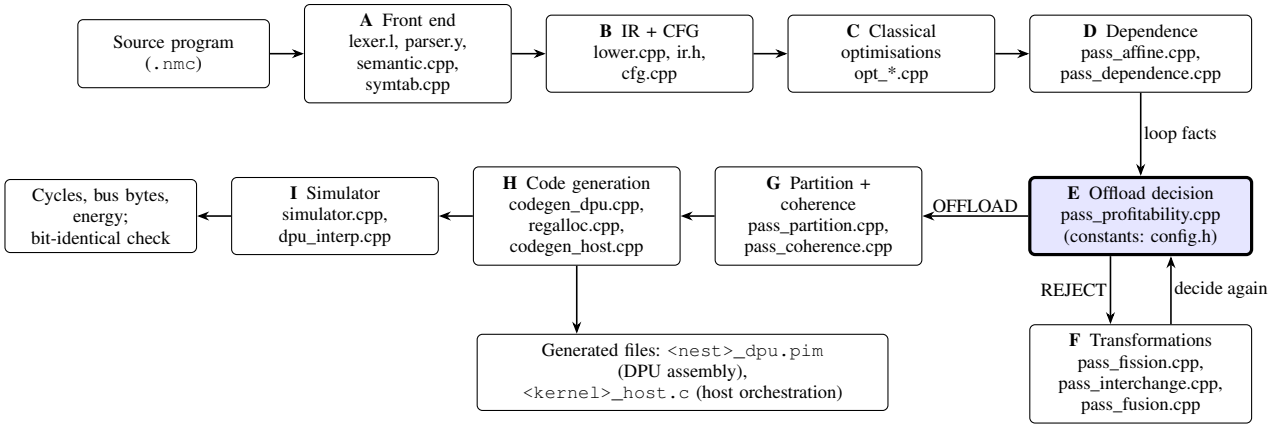


FIGURE 1. Architecture of NEMO-C. Phases A–D analyse the program, phase E decides for every loop nest whether it runs on the host or the DPUs, phase F rewrites rejected nests so they can be decided again, and phases G–I generate, partition and validate the code. Rejected nests stay in the host code.

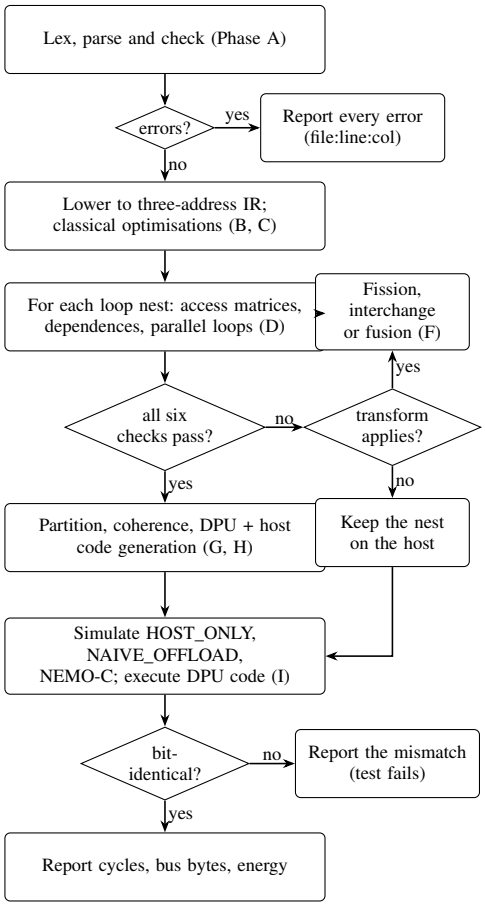


FIGURE 2. Flow of a program through NEMO-C. Every loop nest is decided on its own; a rejected nest may be transformed and decided again, and the simulator checks that the final program is bit-identical to the host-only one.

The simulator closes the loop by checking that the program computes exactly what the host-only program computes.

E. DETAILED METHODOLOGY

1) Front End and Intermediate Representation

The source language has array and scalar declarations, `kernel` blocks that may be offloaded and `host` blocks that always run on the CPU, `for` loops with constant bounds and a positive step, `if`/`else`, assignment, `call` and `print`. The grammar is LALR(1) with zero shift/reduce and reduce/reduce conflicts; braces are mandatory on both arms of an `if`, so the dangling-else ambiguity cannot arise, and operator precedence is declared explicitly. The rule `stmt : error ';'` gives panic-mode recovery. The semantic pass checks that names are declared, that subscripts match the declared dimensions, that loop bounds are integer constants, that a loop index is not reassigned in its body, and that `print` and `call` appear only in host code, a context-sensitive rule that the context-free grammar cannot express.

Lowering produces three-address code in which every statement performs one load, store, operation or copy, with temporaries `%t0`, `%t1`, and so on. Unlike a textbook compiler, NEMO-C keeps loops and conditionals structured in the IR, because dependence analysis, the cost model and the loop transformations all need to see whole loop nests. A flat control-flow graph is built from the same IR whenever a dataflow pass needs one. Five classical passes then run in a fixed order: constant propagation over the lattice {UNDEF, CONST, NAC}, local common-subexpression elimination using available expressions, dead-code elimination driven by backward liveness, induction-variable recognition, and strength reduction that replaces $i \cdot c$ by an accumulator incremented by c. The last matters for PIM because a DPU multiply costs 16 cycles and an addition 2.

2) Dependence Analysis

Every array reference in a nest of depth n is written in affine form

$$f(\mathbf{i}) = F\,\mathbf{i} + \mathbf{c}, \tag{1}$$

where $\mathbf{i} = (i_1, \ldots, i_n)^\top$ is the vector of loop indices, F is the integer access matrix with one row per array dimension and one column per loop, and $\mathbf{c}$ is the offset vector. For

`A[2i+1]`, $F = (2)$ and $\mathbf{c} = (1)$. A subscript that depends on loaded data, such as `A[B[i]]`, has no affine form; the nest is then marked not analysable, unless the reference is a recognised histogram update.

Two references to the same array, at least one of them a write, conflict in dimension d only if the equation $\sum_k a_k i_k - \sum_k b_k i'_k = b_0 - a_0$ has a solution with both iteration vectors inside the loop bounds. The GCD test states that an integer solution can exist only if

$$\gcd(a_1, \ldots, a_n, b_1, \ldots, b_n) \mid (b_0 - a_0), \tag{2}$$

and the Banerjee test states that a real solution can exist only if

$$\sum_{k=1}^{n} \min_{\Omega_k}(a_k i_k - b_k i'_k) \;\le\; b_0 - a_0 \;\le\; \sum_{k=1}^{n} \max_{\Omega_k}(a_k i_k - b_k i'_k), \tag{3}$$

where Ω_k is the region of (i_k, i'_k) allowed by the loop bounds and the tested direction ($<$, $=$ or $>$). Because each term is linear, its extremes are reached at the vertices of Ω_k. The two tests are reported separately because each proves cases the other cannot: `A[2i]` against `A[2i+1]` is independent by the GCD test alone, while `B[i]` against `B[i+100]` with $0 \le i < 64$ is independent only by the Banerjee test. Every direction vector that both tests allow becomes a dependence, classified as flow, anti or output. A loop is parallel exactly when it carries no dependence. Sums, products, conditional maxima and minima, array accumulations and histogram updates are recognised as reductions, so a loop that carries only reduction dependences runs in parallel with private accumulators that the host merges.

3) The Offload Cost Model

For a nest L, NEMO-C counts the arithmetic operations $\mathrm{FLOP}(L)$, multiplying each operation by the trip counts of the loops around it; comparisons and address arithmetic such as the $i - 1$ in `A[i-1]` are excluded, because they are not work on the data. Bytes are counted per stream s, that is, per group of references to the same array with the same access matrix and direction:

$$B(L) = \sum_{s} |\mathrm{fp}(s)| \cdot \beta(s), \tag{4}$$

where $|\mathrm{fp}(s)|$ is the number of distinct elements the stream touches (its footprint, computed exactly by enumeration), and $\beta(s)$ is the number of bytes each element costs on the bus: the element size when the access is contiguous or the working set fits the 32 KB host cache, and $\min(\mathrm{stride}, 64)$ when the stride exceeds the element size and the data do not fit. Re-reads that a cache would capture therefore add nothing to $B(L)$, which is exactly why reuse-heavy kernels such as matrix multiplication come out with a high intensity. The arithmetic intensity follows the roofline model:

$$\mathrm{AI}(L) = \frac{\mathrm{FLOP}(L)}{B(L)}. \tag{5}$$

A stream has self-temporal reuse when some loop's column of F is entirely zero, so that loop re-reads the same element, which a host cache would capture. Read literally, a rule that forbids any reuse would reject matrix–vector multiplication, whose small vectors are reused although the matrix dominates the traffic. NEMO-C therefore counts reuse only when the stream carries a significant share of the bytes, at least $\rho = 0.10$ of them:

$$\mathrm{REUSE}(L) \iff \exists\, s :\ \mathrm{temporal}(s) \ \wedge\ \frac{B(s)}{B(L)} \ge \rho. \tag{6}$$

Finally, a nest that finishes on the host before the DPUs could even be started cannot gain from offloading. NEMO-C estimates the host time as one DRAM line fill per distinct cache line, one cache hit per array access and one cycle per operation, and compares it with the cost that PIM pays even for no work, namely the launch overhead and the transfer of the same bytes:

$$\begin{aligned} \hat{T}_{\mathrm{host}}(L) &= \tfrac{B(L)}{64}\, c_{\mathrm{miss}} + A(L)\, c_{\mathrm{hit}} + \mathrm{FLOP}(L)\, c_{\mathrm{alu}}, \\ T^{\mathrm{fix}}_{\mathrm{pim}}(L) &= c_{\mathrm{launch}} + B(L)\, c_{\mathrm{dma}}, \end{aligned} \tag{7}$$

where $A(L)$ is the number of array accesses and the constants are those of Table 3 ($c_{\mathrm{miss}} = 100$, $c_{\mathrm{hit}} = 4$, $c_{\mathrm{alu}} = 1$, $c_{\mathrm{launch}} = 80{,}000$, $c_{\mathrm{dma}} = 0.5$). The decision predicate is the conjunction of six checks:

$$\begin{aligned} \mathrm{OFFLOAD}(L) \iff\ & \mathrm{AI}(L) < 0.5 \ \wedge\ \neg\mathrm{REUSE}(L) \\ & \wedge\ \mathrm{PAR}_{\mathrm{outer}}(L) \ \wedge\ \mathrm{PART}(L) \\ & \wedge\ \mathrm{class}(L) \neq \mathrm{IRREGULAR} \\ & \wedge\ \hat{T}_{\mathrm{host}}(L) > T^{\mathrm{fix}}_{\mathrm{pim}}(L), \end{aligned} \tag{8}$$

where $\mathrm{PAR}_{\mathrm{outer}}$ holds when the outermost loop is parallel or reduction-parallel and PART when the nest can be partitioned without an all-to-all exchange (Section III-E5). Floating-point reductions across DPUs are also rejected, because adding partial sums in a different order would break the bit-identical guarantee. The class of a nest (ELEMENTWISE, REDUCTION, STENCIL, COMPUTE_BOUND or IRREGULAR) is derived from the same direction vectors and access matrices, never from the syntax of the loop.

Algorithm 1 gives the procedure and Fig. 3 its flowchart. All six checks are evaluated, not only the first failing one, so that the trace printed by `--explain` lists every reason. For matrix multiplication (16×16) the trace reports 8,192 operations over 4,096 bytes, so AI $= 2.0$; each of the four streams carries 25% of the traffic with self-temporal reuse; and the host estimate of 80,128 cycles does not exceed the PIM fixed cost of 82,048. Three checks fail and the nest stays on the host. For saxpy over 16,384 elements, $\mathrm{AI} = 32{,}768/196{,}608 = 0.17$, no stream has reuse, the loop is parallel and the host estimate of 536,576 cycles exceeds the fixed cost of 178,304, so the nest is offloaded.

4) Enabling Transformations

When a nest is rejected, three classical transformations are tried, each checked for legality on the dependence graph, and

Algorithm 1 Six-check offload decision for one loop nest

Require: nest L with affine references, dependences D, constants of Table 3
Ensure: verdict $v \in \{\text{OFFLOAD}, \text{REJECT}\}$ and the list of failed checks R

$R \leftarrow \emptyset$
if some reference of L is non-affine and not a histogram update **then**
  $R \leftarrow R \cup \{\text{IRREGULAR}\}$
else
  compute $\text{FLOP}(L)$, $B(L)$ by (4) and $\text{AI}(L)$ by (5)
  if $\text{AI}(L) \geq 0.5$ **then**
    $R \leftarrow R \cup \{\text{compute-bound}\}$
  end if
  if $\text{REUSE}(L)$ holds by (6) **then**
    $R \leftarrow R \cup \{\text{reuse}\}$
  end if
  if the outermost loop carries a non-reduction dependence in D **then**
    $R \leftarrow R \cup \{\text{serial}\}$
  end if
  if a reduction across DPUs is in floating point **then**
    $R \leftarrow R \cup \{\text{reassociation}\}$
  end if
  if $\hat{T}_{\text{host}}(L) \leq T^{\text{fix}}_{\text{pim}}(L)$ by (7) **then**
    $R \leftarrow R \cup \{\text{too small}\}$
  end if
  if $R = \emptyset$ **and** owner-computes partitioning of L fails **then**
    $R \leftarrow R \cup \{\text{all-to-all}\}$
  end if
end if
if $R = \emptyset$ **then**
  $v \leftarrow$ OFFLOAD
else
  $v \leftarrow$ REJECT
end if
return (v, R)

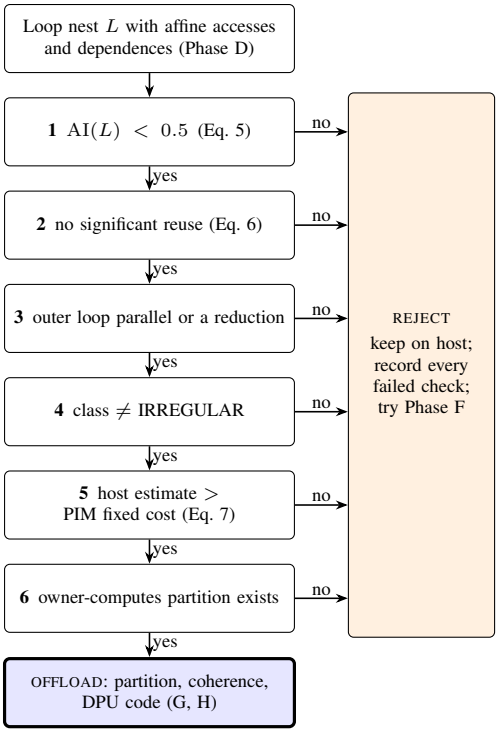


FIGURE 3. Offload decision of Phase E (Algorithm 1). All checks are evaluated so that the explanation lists every reason; a nest is offloaded only if all six hold.

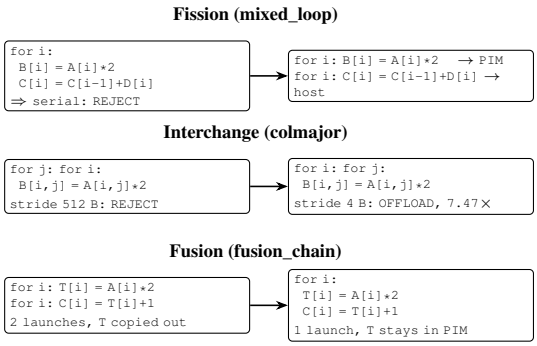


FIGURE 4. The three enabling transformations on the benchmarks where they apply. Each is checked for legality on the dependence graph, and the result is decided again.

the resulting nests are decided again by Algorithm 1. Fig. 4 shows one example of each.

Loop fission follows Allen and Kennedy: the statements of the loop form a graph whose edges are the dependences between them, Tarjan's algorithm finds its strongly connected components, and the components are emitted in topological order, one loop each. In `mixed_loop`, a parallel statement `B[i]=A[i]*2` shares a loop with the recurrence `C[i]=C[i-1]+D[i]`; fission separates them, and the first loop is offloaded while the second stays on the host.

Loop interchange permutes the loops of a nest; a permutation is legal when every direction vector remains lexicographically non-negative. In `colmajor`, the inner loop walks down a column with a 512-byte stride and the outer loop does not follow the partitioned dimension; swapping the loops makes the inner access contiguous and the outer loop partitionable, and the counted memory traffic falls from 2,097,152 to 131,072 bytes.

Loop fusion merges adjacent loops with the same bounds when no dependence would point backwards in the fused loop. In `fusion_chain`, the intermediate array `T` is produced by one loop and consumed by the next; after fusion it never leaves the DPUs, which saves one launch (80,000 cycles) and 131,072 bytes of DMA.

5) Partitioning and Coherence

An offloaded nest is divided among the DPUs by the owner-computes rule: each DPU owns a contiguous block of rows of every array (dimension 0) and executes exactly the iterations that write rows it owns; with 16,384 iterations and 64 DPUs

each DPU receives 256. Every read is then classified as local (same access function), halo (a constant offset δ on the partitioned dimension, which needs $|\delta|$ neighbouring rows), broadcast (a small array every DPU needs), private (a reduction accumulator, merged afterwards) or impossible (a write whose row does not follow the partitioned loop). The last case is the canonical failure of transpose: every DPU would write a column of `B[j,i]` spread over all other DPUs' rows, an all-to-all exchange that the hardware cannot perform. Block, cyclic and block-cyclic distributions are scored by halo bytes plus a fixed cost per separate transfer run, and the cheapest is chosen; for stencil1d the scores are 77,304 for block, 1,236,992 for block-cyclic and 19,789,456 for cyclic.

The host's cache and the DPUs' copies of an array can disagree, so NEMO-C inserts `flush(X)` before a kernel reads an array the host has written and `invalidate(X)` before the host reads an array a kernel has written. The placement is a forward dataflow analysis over the host CFG, whose facts are pairs of sets of arrays: H (written by the host, not yet flushed) and P (written on the DPUs, not yet invalidated in the host cache). The meet over predecessors is set union and the transfer function of a block b is applied statement by statement:

$$\begin{aligned} \mathrm{IN}(b) &= \textstyle\bigcup_{p\in\mathrm{pred}(b)} \mathrm{OUT}(p), \\ \mathrm{OUT}(b) &= \big(\mathrm{IN}(b) \setminus \mathrm{KILL}(b)\big) \cup \mathrm{GEN}(b), \end{aligned} \tag{9}$$

where a host store of X generates X in H, a kernel that reads $X \in H$ requires a flush and kills X from H, a kernel that writes X generates X in P, and a host load of $X \in P$ requires an invalidate and kills X from P. Algorithm 2 iterates (9) to a fixed point and then inserts the operations. Union is the safe meet: if an array is stale on any path into a block, the block must treat it as stale. Fig. 5 shows the effect on a program in which the host reads the kernel's output inside one branch of an `if` and again after it. The path that skips the branch still carries $C \in P$, so the read after the merge needs its own invalidate. The analysis reaches its fixed point in three rounds, because the loop's back edge carries $H = \{A\}$ to the loop header in the second round.

6) Code Generation and Register Allocation

For each offloaded nest the back end selects instructions of a small PIM instruction set (`LD`, `ST`, `ADD`, `SUB`, `MUL`, `DIV`, `MOVI`, `DMA_IN`, `DMA_OUT`, `LOOP`, `ENDLOOP` and `HALO_EXCHANGE`, extended with comparisons, `MOV`, `MIN`, `ARG`/`RET` and floating-point variants so that conditionals and reductions can be expressed). Because a DPU computes only from its 64 KB scratchpad, the generated code processes its slice in tiles of T iterations, with

$$T = \left\lfloor \frac{W - W_{\mathrm{spill}} - W_{\mathrm{res}} - W_{\mathrm{halo}}}{b_{\mathrm{iter}}} \right\rfloor, \tag{10}$$

where $W = 65{,}536$ bytes, W_{spill} is the spill area, W_{res} holds broadcast arrays, W_{halo} the halo rows, and b_{iter} the bytes one iteration moves; for stencil1d, $T = \lfloor(65{,}536 - 512 - 0 - 8)/8\rfloor = 8{,}127$. The arithmetic is written as a comment at the top of every generated file.

Algorithm 2 Placement of flush and invalidate operations

Require: host CFG $G = (V, E)$; for each kernel call, the arrays it reads and writes
Ensure: IR with `flush` and `invalidate` inserted

for all $b \in V$ **do**
  $\mathrm{OUT}(b) \leftarrow (\emptyset, \emptyset)$
end for
repeat
  $changed \leftarrow$ **false**
  for all $b \in V$ in reverse post-order **do**
    $(H, P) \leftarrow \bigcup_{p\in\mathrm{pred}(b)} \mathrm{OUT}(p)$
    for all statement s in b **do**
      if s stores array X **then**
        $H \leftarrow H \cup \{X\}$
      end if
      if s calls a kernel **then**
        $H \leftarrow H \setminus \mathrm{reads}(s)$; $P \leftarrow P \cup \mathrm{writes}(s)$
      end if
      if s loads array $X \in P$ **then**
        $P \leftarrow P \setminus \{X\}$
      end if
    end for
    if $(H, P) \neq \mathrm{OUT}(b)$ **then**
      $\mathrm{OUT}(b) \leftarrow (H, P)$; $changed \leftarrow$ **true**
    end if
  end for
until not $changed$
replay each block from its fixed-point IN set; insert `flush(X)` where a call reads $X \in H$ and `invalidate(X)` where a load reads $X \in P$

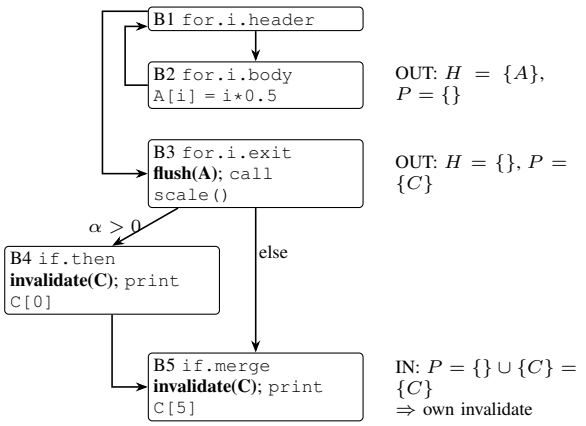


FIGURE 5. Coherence analysis on `coherence_demo`. At the merge block B5 the union of $P = \{\}$ (through B4) and $P = \{C\}$ (skipping B4) is $\{C\}$, so the second read of `C` needs its own invalidate.

Registers are allocated by Chaitin–Briggs graph colouring with $K = 24$ physical registers (Algorithm 3, Fig. 6). Liveness is computed by a backward dataflow analysis over the DPU program, $\mathrm{in}(n) = \mathrm{use}(n) \cup (\mathrm{out}(n) \setminus \mathrm{def}(n))$, and two virtual registers interfere when one is defined while

Algorithm 3 Chaitin–Briggs register allocation

Require: DPU program over virtual registers; $K = 24$
Ensure: colouring $\kappa : V \to \{1, \dots, K\}$ with $\kappa(u) \neq \kappa(v)$ for every interference edge

repeat
  compute liveness; build interference graph $G = (V, E)$
  coalesce copy pairs that pass the Briggs test
  $S \leftarrow$ empty stack
  while $V \neq \emptyset$ **do**
    if some v has $\deg(v) < K$ **then**
      push v on S; remove v
    else
      push $\arg\min_v \mathrm{prio}(v)$ by (11); remove it
    end if
  end while
  $spilled \leftarrow \emptyset$
  while S not empty **do**
    pop v; give it the lowest colour unused by coloured neighbours
    if no colour is free **then**
      $spilled \leftarrow spilled \cup \{v\}$
    end if
  end while
  insert spill loads and stores for every $v \in spilled$
until $spilled = \emptyset$
return κ

the other is live. Copy-related registers are merged first by the Briggs conservative test, which allows a merge only if the merged node has fewer than K neighbours of degree at least K, so coalescing can never make a colourable graph uncolourable. Nodes of degree below K are then removed onto a stack; when none remains, the node with the lowest spill priority

$$\mathrm{prio}(v) = \frac{\sum_{x \in \mathrm{defs}(v) \cup \mathrm{uses}(v)} 10^{\,\mathrm{depth}(x)}}{\deg(v)} \tag{11}$$

is pushed optimistically. Popping the stack assigns each node the lowest colour not used by its neighbours; a node that finds none is spilled to the scratchpad with a load before each use and a store after each definition, and the whole process repeats. Finally, the host side is generated as C code that flushes, splits the iterations among the 64 DPUs, transfers each DPU's rows with their halo, launches and waits for the DPUs, copies the results back and merges reductions.

7) Simulation and Validation

Because no PIM hardware was available, NEMO-C includes a simulator that both executes and costs the program. The host model is a 32 KB, 8-way set-associative LRU cache with 64-byte lines; a hit costs 4 cycles, a miss 100, and each line fill or write-back puts 64 bytes on the bus. For an offloaded

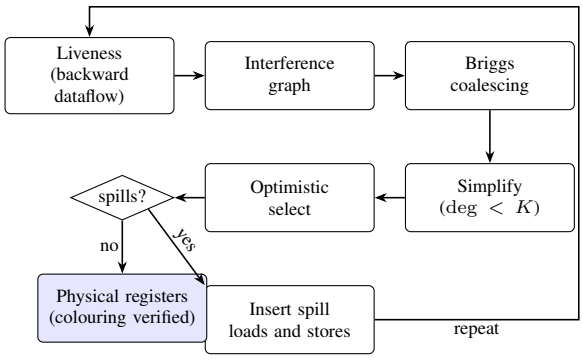


FIGURE 6. Chaitin–Briggs register allocation (Algorithm 3). Spilling changes the program, so the process repeats until every virtual register has a physical register.

nest the simulated time is

$$T_{\mathrm{pim}} = c_{\mathrm{launch}} + \underbrace{c_{\mathrm{dma}} B + c_{\mathrm{run}} N_{\mathrm{run}} + c_{\mathrm{set}} N_{\mathrm{set}}}_{\text{DMA in and out}} + \max_{d} T_{\mathrm{dpu}}(d) + T_{\mathrm{merge}}, \tag{12}$$

where N_{run} counts separate contiguous transfer segments, N_{set} the per-array transfer sets, and $T_{\mathrm{dpu}}(d)$ the cycles of DPU d: 2 per ALU operation, 16 per multiply, 32 per divide, and 10 per scratchpad access divided by the overlap of $\min(16, 11)$ tasklets and multiplied by a bank-conflict factor for strides that are multiples of the bank count. Each DPU sees a private image of the arrays in which only the rows the partition assigned to it are present, so a wrong partition or halo is detected as an access error, and two DPUs writing the same element are reported as a race. The results of every configuration are compared bit for bit with the host-only run, the unoptimised IR is run once more to check the optimiser, and the generated, register-allocated DPU programs are executed by an interpreter (`dpu_interp.cpp`) and must agree as well. The speedup of a configuration is

$$S_{\mathrm{cfg}} = \frac{T_{\mathrm{host_only}}}{T_{\mathrm{cfg}}}. \tag{13}$$

IV. IMPLEMENTATION AND RESULT ANALYSIS

A. TOOLS, CONFIGURATION AND SETUP

NEMO-C is written in C++17 using only the standard library. The scanner is generated by Flex 2.6.4 and the LALR(1) parser by GNU Bison 3.8.2 [28]; the compiler, the test programs and the generated host code are built with GCC (g++ 15.2) [29] through a plain GNU Make 4.4.1 build. The experiments ran under Ubuntu 26.04 LTS in WSL2 on a laptop with a 13th-generation Intel Core i7-1360P processor (16 hardware threads) and 7 GB of memory available to the Linux environment. Plots are drawn by a separate Python script with Matplotlib [30], kept outside the compiler, and a browser workbench served by a Python standard-library server drives the same compiler binary for demonstration.

The project is built with `make`; `./nemoc prog.nmc --decisions` prints one verdict per nest, `--explain <kernel>` prints the full decision trace, `--codegen`

TABLE 3. Parameters of the machine model (`config.h`).

Parameter	Value
DPUs P; tasklets per DPU	64; 16
Scratchpad (WRAM) per DPU	65,536 bytes
DPU registers K	24
Host cache hit / miss	4 / 100 cycles
Host ALU operation	1 cycle
DPU ALU / multiply / divide	2 / 16 / 32 cycles
DPU scratchpad access	10 cycles
DMA per byte	0.5 cycle
Machine balance	0.5 flop/byte
Cache line	64 bytes
Energy: DRAM line / PIM access / ALU	640 / 60 / 1 pJ
Added to complete the model	
Launch and synchronisation overhead	80,000 cycles
DMA setup per transfer set / per run	2,000 / 300 cycles
Host cache size and associativity	32 KB, 8-way LRU
Tasklets that saturate the pipeline	11
Memory banks per DPU	8
Reuse significance threshold ρ	0.10

writes the DPU assembly and host C, `--dump regalloc --regs K` prints the allocation for a register budget K, and `--simulate` runs the three configurations. The command `make test` builds and runs the eight test programs, and `make run-all` simulates all benchmarks and writes `results/results.csv`, from which every number in this section is taken. Fig. 7 shows the workbench executing `--explain` on matrix multiplication.

B. PARAMETER SETUP

Table 3 lists the parameters of the machine model. The first group is given by the course specification of the target; the second was added to complete the model, and each value is justified in the source file `config.h`. All benchmarks use 16,384 one-dimensional elements or 128×128 matrices, except matrix multiplication (16×16) and the histogram (256 bins over 16,384 inputs).

C. BENCHMARKS AND DECISIONS

Table 4 lists the fifteen benchmarks, their class and arithmetic intensity as computed by Phase E, and the verdict. They were chosen to cover every case the decision must handle: streaming element-wise kernels, reductions, stencils, a matrix–vector product with negligible reuse, three kernels that need one of the enabling transformations, and three that must be rejected for three different reasons.

Three configurations are simulated for each benchmark. HOST_ONLY runs everything on the CPU. NAIVE_OFFLOAD offloads every parallel loop without consulting the cost model and without the enabling transformations, which is what a programmer obtains by porting every parallel kernel to the DPUs. NEMO-C applies Algorithm 1 and the transformations. We report six performance metrics: execution time (as speedup), memory-bus traffic, energy, decision correctness, register pressure and compile time, together with the correctness checks.

TABLE 4. Benchmarks and NEMO-C verdicts (AI in flop/byte).

Benchmark	Class	AI	Verdict and reason
vecadd	elementwise	0.08	OFFLOAD
saxpy	elementwise	0.17	OFFLOAD
dotprod	reduction	0.25	OFFLOAD (integer sum)
maxreduce	reduction	0.00	OFFLOAD (conditional max)
histogram	reduction	0.24	OFFLOAD (private histograms)
gemv	elementwise	0.49	OFFLOAD (reuse 1% of bytes)
stencil1d	stencil	0.37	OFFLOAD (halo 1 row)
stencil2d	stencil	0.49	OFFLOAD (halo)
filter	elementwise	0.00	OFFLOAD (data-dependent if)
fusion_chain	elementwise	0.12	OFFLOAD after fusion
mixed_loop	mixed	0.12	1 of 2 loops after fission
colmajor	elementwise	0.12	OFFLOAD after interchange
matmul	compute-bound	2.00	REJECT: AI, reuse, size
transpose	elementwise	0.00	REJECT: all-to-all partition
indirect	irregular	n/a	REJECT: non-affine `A[B[i]]`

D. METRIC 1: EXECUTION TIME

Fig. 8 shows the speedup of both offloading configurations over HOST_ONLY. NEMO-C is never slower than the host. For the nine benchmarks that offload as written, NEMO-C and NAIVE_OFFLOAD make the same choice and gain between $1.01\times$ (histogram, where merging 64 private histograms on the host absorbs most of the benefit) and $1.68\times$ (stencil2d, where the host performs several reads per element). The transformations give the largest gains: $7.47\times$ for colmajor after interchange, against $0.71\times$ when the column-order loop is offloaded unchanged; $2.08\times$ for fusion_chain after fusion, against $1.46\times$; and $1.12\times$ for mixed_loop after fission, where blind offloading cannot offload anything. On the three rejected benchmarks NAIVE_OFFLOAD runs at $0.63\times$ (matmul), $0.70\times$ (transpose) and $0.43\times$ (indirect), while NEMO-C matches the host exactly. Over all fifteen benchmarks the total simulated time is 15.48 M cycles for HOST_ONLY, 17.49 M for NAIVE_OFFLOAD ($0.89\times$) and 10.20 M for NEMO-C ($1.52\times$). The justification is the cost model itself: the rejected nests are precisely those in which launch, DMA and lost cache reuse cost more than near-memory execution saves.

E. METRIC 2: MEMORY-BUS TRAFFIC

Fig. 9 shows the bytes that cross the memory bus, normalised to HOST_ONLY. For streaming kernels offloading moves roughly the same bytes as the host (0.84–$1.33\times$), because every operand is copied in and every result copied out once. The differences appear where locality differs. NEMO-C moves $0.08\times$ the host's bytes on colmajor, because the host drags a 64-byte line per element along a column while the transformed DPU code moves contiguous rows, and $0.52\times$ on fusion_chain, because the intermediate array never leaves PIM. Blind offloading of matmul moves $8.4\times$ the host's bytes, because the whole of `B` is broadcast to every DPU while the host serves it from its cache, and of indirect $6.4\times$, because every DPU needs all of `A`. Transpose shows that

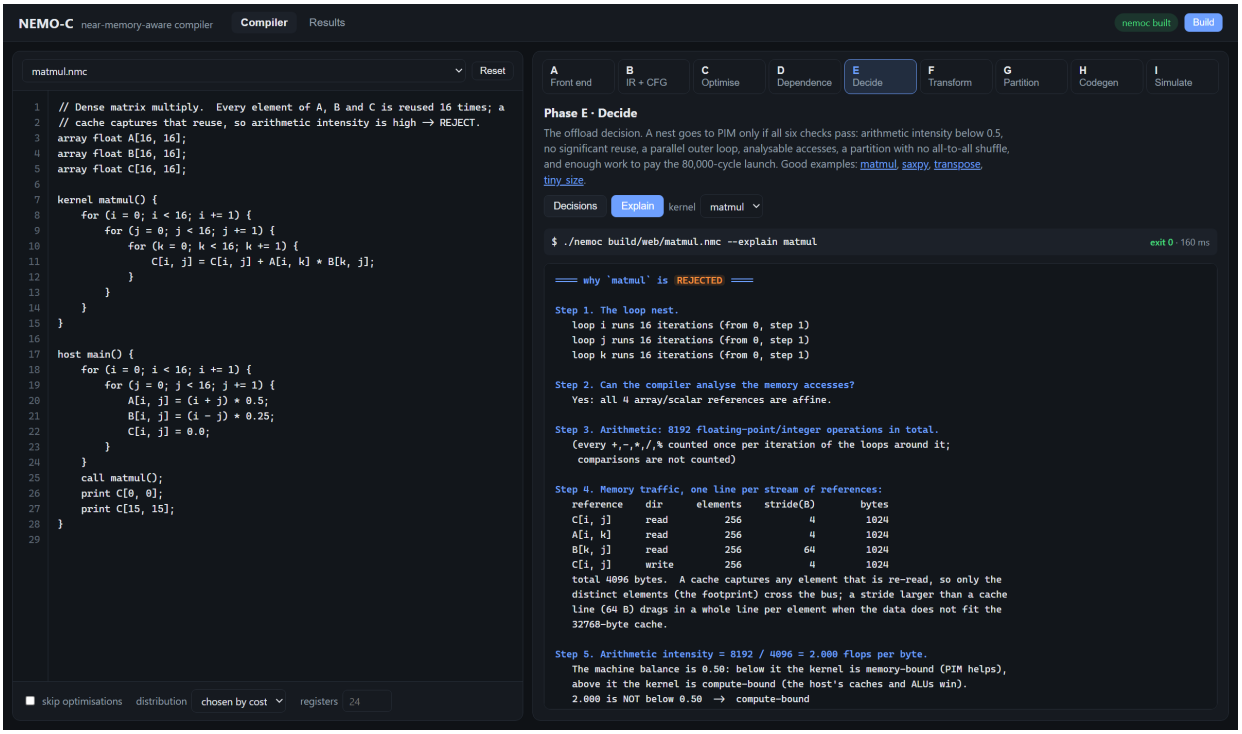


FIGURE 7. The NEMO-C workbench running `./nemoc matmul.nmc --explain matmul`. The left pane holds the program; the right pane shows phases A–I and the compiler's decision trace, which reports $\mathrm{AI} = 8192/4096 = 2.0$ and rejects the nest.

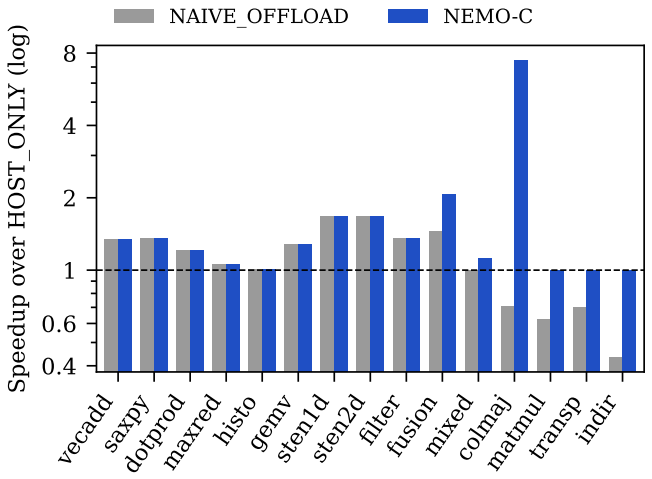


FIGURE 8. Metric 1: speedup over HOST_ONLY (log scale; Eq. 13). NEMO-C never falls below 1.0; blind offloading loses on colmajor, matmul, transpose and indirect.

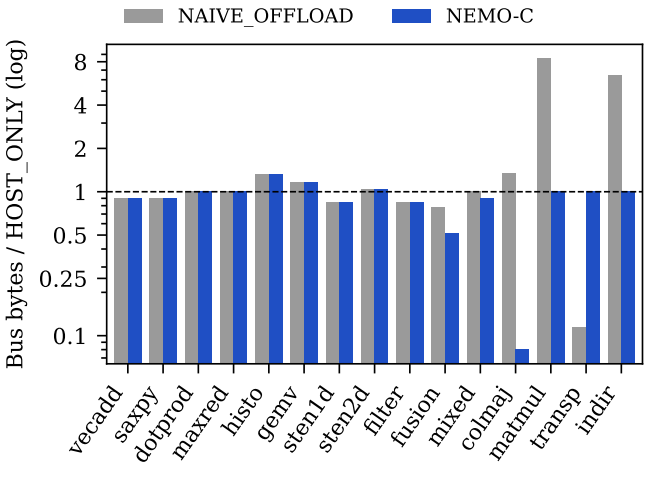


FIGURE 9. Metric 2: memory-bus bytes normalised to HOST_ONLY (log scale).

traffic alone is not the criterion: blind offloading moves only 0.12× the bytes, yet runs 0.70× slower, because its output is split into 8,192 separate transfer segments. In total NEMO-C moves 6.95 MB against 10.35 MB for the host and 13.15 MB for blind offloading.

F. METRIC 3: ENERGY

Fig. 10 shows energy normalised to HOST_ONLY. Over the whole set NEMO-C uses 73.3 μJ against 105.1 μJ for the host (0.70×) and 143.4 μJ for blind offloading (1.36×). Most of the saving comes from colmajor (0.09×) and fusion_chain (0.56×), and the rejections avoid the 6.9× energy of blind offloading on matmul and indirect. Two results are less favourable and we report them as measured. Several offloaded streaming kernels use slightly more energy than the host (up to 1.39× for histogram and 1.23× for gemv), because each DMA burst costs a DRAM access plus a bank access. And transpose under blind offloading uses less energy than the host (0.13×) although it is slower. NEMO-C's rule

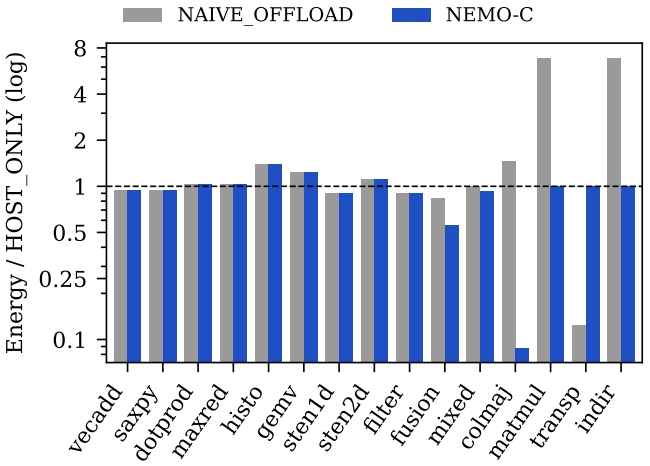


FIGURE 10. Metric 3: energy normalised to HOST_ONLY (log scale).

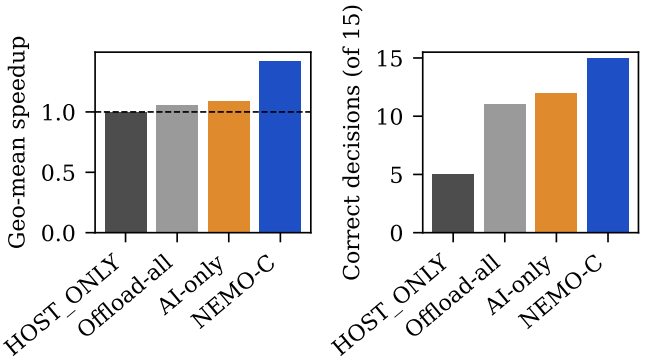


FIGURE 11. Metric 4: geometric-mean speedup and number of decisions that are no worse than the better of HOST_ONLY and NAIVE_OFFLOAD, for four offload policies over the 15 benchmarks.

optimises time, not energy, which is a limitation discussed below.

G. METRIC 4: DECISION CORRECTNESS AND COMPARISON WITH RELATED WORK

We compare NEMO-C with three related works by implementing their offload policies in the same simulator, since their tool chains do not accept our source language. The offload-all policy corresponds to the hand-ported workloads of PrIM [13] and to programmer-directed frameworks such as SimplePIM [16], in which every kernel the programmer marks runs on PIM; it is our NAIVE_OFFLOAD configuration. The AI-only policy represents a single-metric roofline rule that offloads whenever AI $<$ 0.5, ignoring reuse, partitionability and size. Devic et al. [14] make selective decisions for bank-level RISC-V cores; their framework is not reproduced quantitatively here, and Table 5 compares it qualitatively. A decision is counted as correct when its time is no worse than the better of HOST_ONLY and NAIVE_OFFLOAD for that benchmark.

Fig. 11 and Table 5 give the outcome. Offload-all reaches a geometric-mean speedup of 1.06× with four slowdowns (worst 0.43×) and 11 of 15 correct decisions; AI-only reaches 1.09× with three slowdowns and 12 correct decisions, because it rejects matmul but cannot see the all-to-all partition of transpose, the irregularity of indirect or the stride of colmajor. NEMO-C reaches 1.42× with no slowdown and 15 of 15 correct decisions, and on three benchmarks it exceeds both simple choices because the transformations create options that neither policy has.

TABLE 5. Comparison with related work. Quantitative rows use the same simulator and 15 benchmarks.

	Offload-all [13], [16]	**AI-only roofline**	**Devic *et al.* [14]**	**NEMO-C**
Who decides	programmer	one metric	framework	cost model
Refuses offloads	no	partly	yes	yes
Loop transforms	no	no	not reported	yes
Explains verdict	–	–	not reported	yes
Geo-mean speedup	1.06×	1.09×	–	**1.42×**
Slowdowns (of 15)	4	3	–	**0**
Worst speedup	0.43×	0.43×	–	**1.00×**
Correct decisions	11	12	–	**15**

H. METRIC 5: REGISTER PRESSURE

Fig. 12 shows how many virtual registers the Chaitin–Briggs allocator spills as the register budget K shrinks, for the three offloaded kernels with the largest DPU programs. With the default $K = 24$ no kernel spills; stencil2d and gemv need 17 registers and stencil1d 14, which leaves headroom for larger loop bodies. Spills grow roughly linearly as K falls, reaching

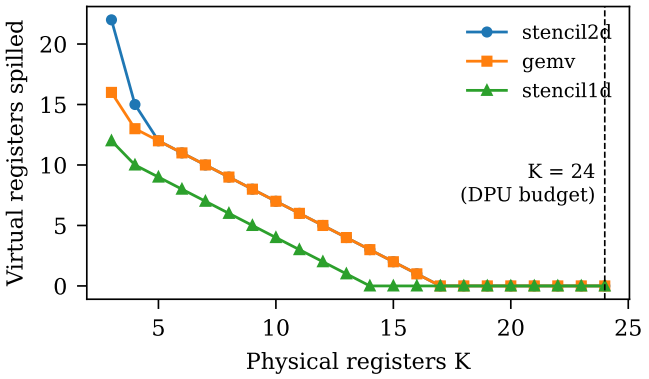


FIGURE 12. Metric 5: virtual registers spilled by the Chaitin–Briggs allocator as the register budget K varies.

22 for stencil2d at $K = 3$, and in every case the colouring was verified to assign different registers to all interfering pairs and the spilled programs remained bit-identical when executed. The generated DPU programs contain 42 to 80 instructions, with tile sizes between 62 and 16,256 iterations depending on the bytes each iteration moves.

I. METRIC 6: COMPILE TIME

Fig. 13 shows the time to compile each benchmark from source to DPU assembly and host C (median of seven runs). The mean is 42.1 ms and the maximum 59.9 ms (stencil2d).

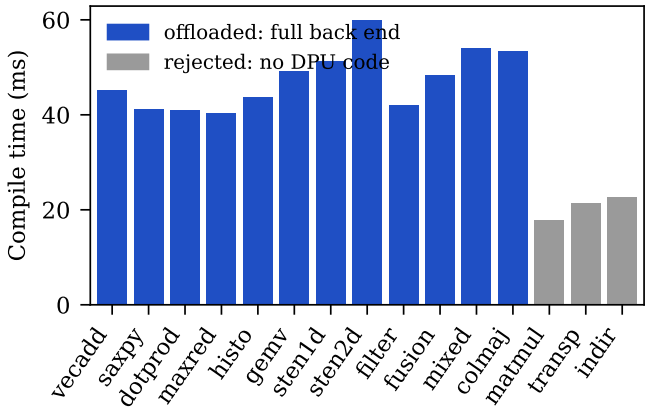


FIGURE 13. Metric 6: compile time per benchmark, from source to generated code (median of seven runs).

The rejected kernels compile in 18–23 ms because no DPU code is generated for them. The analyses behind the decision, including footprint enumeration and the enumeration of direction vectors, therefore add negligible cost to compilation.

J. ABLATION OF THE DECISION CHECKS

To see whether every check of (8) earns its place, we removed each in turn and asked which verdict would change and with what effect, using the failed-check lists that Algorithm 1 records and the simulated times of the affected programs. Table 6 gives the outcome. Removing the arithmetic-intensity check or the reuse check alone changes no verdict on this suite, because matrix multiplication fails three checks and either of the remaining two still rejects it; the checks overlap deliberately, so that a compute-bound kernel is caught for more than one reason. The reuse threshold matters in the other direction: a literal rule that forbids any self-temporal reuse would reject gemv and lose its 1.29× speedup. The irregularity, partitioning and size checks each prevent a real loss, and the parallelism check prevents a wrong result rather than a slow one, since it is the only guard against running the iterations of a recurrence concurrently. The size check was motivated by a 256-element element-wise loop (`tests/tiny_size.nmc`) that passes every other check; the host finishes it in 7,428 simulated cycles, while blind offloading needs 128,383 (0.06×), because the 80,000-cycle launch alone exceeds the host's total time. With the check, the compiler estimates 5,760 host cycles against a fixed PIM cost of 81,024 and keeps the loop on the host.

K. CORRECTNESS

All 45 benchmark–configuration runs produce results bit-identical to HOST_ONLY, the unoptimised IR gives the same results as the optimised IR for every benchmark, and the generated, register-allocated DPU programs of all fifteen benchmarks execute with identical results in the interpreter. The test suite of eight programs and 365 checks passes, and the simulator's dynamic coherence checker reports no stale read. The last property also shows that the slowdowns of blind offloading are performance effects, not errors.

TABLE 6. Ablation: the effect of removing one check of the decision predicate.

Check removed	Verdict that changes	Effect
AI < 0.5	none (matmul still fails reuse and size)	–
Significant reuse	none (matmul still fails AI and size)	–
Threshold ρ (any reuse rejects)	gemv rejected	1.29× lost
Parallel outer loop	serial recurrences offloaded	wrong results (by construction; not simulated)
Not IRREGULAR	indirect offloaded	1.00× →0.43×
Partitionable	transpose offloaded	1.00× →0.70×
Size check	256-element loop offloaded	1.00× →0.06×

L. DISCUSSION AND LIMITATIONS

The results support the central claim that near-memory offloading should be conditional and that a small set of classical analyses is enough to make the condition reliable on this benchmark set. The most instructive cases are the three rejections, because each fails for a different reason that a single metric cannot see. Matrix multiplication is parallel and would be accepted by a parallelism-only rule, yet it is compute-bound and its working set fits the host cache. Transpose has an arithmetic intensity of zero and would be accepted by a roofline rule, yet its writes cross every partition. Indirect access looks memory-bound too, yet its subscripts are unknown at compile time, so every DPU would need a full copy of the indexed array. The predicate rejects all three, and the explanation it prints names the failing check in each case, which makes the decision easy to audit.

Threats to validity. Construct validity depends on the simulator: cycles, bytes and energy are model outputs, and their relation to real hardware rests on the constants of Table 3. Internal validity is helped by determinism, since every run of the same program gives the same numbers and `make run-all` reproduces the published CSV exactly, but the per-benchmark values are single deterministic runs rather than measurements with variance. External validity is limited by the benchmark set, which contains fifteen small, single-kernel programs chosen to exercise each check; larger applications with several interacting kernels may behave differently. Conclusion validity is supported by the comparison of policies on the same simulator and the same inputs, so the relative ordering of the policies does not depend on the absolute calibration.

Several further limitations qualify the claim. The results come from a simulator, not from physical UPMEM hardware, and the constants of Table 3 were chosen to be plausible rather than calibrated; the launch overhead and the per-run DMA cost have the largest influence on small kernels. The cost model optimises time and ignores energy, so it rejects transpose although offloading it would save energy. Data are

copied in and out on every launch; keeping arrays resident across kernels would change several verdicts. Finally, the benchmarks are small, single-kernel programs; fusion currently requires both loops to use the same index variable, and fission refuses loops that contain strength-reduced accumulators.

V. CONCLUSION AND SCOPE FOR FUTURE WORK

This paper presented NEMO-C, a compiler that decides for every loop nest whether it should run on the host or on processing-in-memory DPUs, and that refuses offloads which would make the program slower.

Existing PIM tools characterise the hardware or generate efficient PIM code, but they mostly leave the decision of what to offload to the programmer or confine it to neural-network operators.

NEMO-C makes the decision statically with six checks built from classical analyses, explains every verdict, and uses fission, interchange and fusion to turn rejected loops into profitable ones.

It is implemented from scratch in C++17 with Flex and Bison and evaluated in a cycle-level simulator of an UPMEM-style system.

On fifteen benchmarks NEMO-C is never slower than the host and reaches a geometric-mean speedup of 1.42× and a best case of 7.47×. It avoids the 0.63×, 0.70× and 0.43× slowdowns of blind offloading and makes the correct decision on all fifteen benchmarks, against twelve for an arithmetic-intensity-only rule. It reduces memory-bus traffic to 0.67× and energy to 0.70× of the host's, and every one of its 45 simulated runs, including the execution of its own generated assembly, is bit-identical.

Future work will first calibrate the cost model against measurements on real UPMEM hardware and study the sensitivity of the verdicts to the launch and transfer constants. The decision should become energy-aware, so that a user can trade time for energy. Keeping arrays resident in PIM across kernel calls, and deciding offloads for whole programs rather than isolated nests, would extend the approach to multi-kernel applications. Support for irregular loops, for example by partial offload with inspector–executor techniques, and an exact integer dependence test would widen the set of loops the compiler can accept.

AUTHOR CONTRIBUTIONS

Somesh Tripathi conceived the project, designed and implemented the complete NEMO-C compiler (front end, intermediate representation and classical optimisations, dependence analysis, offload cost model, loop transformations, partitioning and coherence, code generation and register allocation), the simulator, the test suite and the browser workbench, ran all experiments and analysed the results. Somesh Tripathi, Smeehen and Swati Saumya contributed to the literature survey and to writing, editing and reviewing the manuscript. All authors read and approved the final manuscript.

ACKNOWLEDGMENT

The authors thank the School of Computer Science and Engineering, Vellore Institute of Technology, for the resources provided for this Compiler Design project. The AI assistant Claude (Anthropic), used through Claude Code, assisted in this work: it helped write, debug and test parts of the compiler, and also helped with parts of the text, figures and LaTeX source of this manuscript. The authors directed this use, checked every number in the paper against the compiler's own output in `results/results.csv`, verified the cited works, and take full responsibility for the content.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

COMPETING INTERESTS

The authors declare no competing financial or non-financial interests.

DATA AND CODE AVAILABILITY

The complete source code of NEMO-C, the fifteen benchmark programs, the test suite, the simulation results and the scripts that produce every figure of this paper are available at https://github.com/Somesh2223/Near-Memory-Computing-Compiler.

REFERENCES

[1] Mutlu, Onur, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. (2019) "Processing data where it makes sense: Enabling in-memory computation." Microprocessors and Microsystems 67: 28–41.

[2] Singh, Gagandeep, Lorenzo Chelini, Stefano Corda, Ahsan Javed Awan, Sander Stuijk, Roel Jordans, Henk Corporaal, and Albert-Jan Boonstra. (2019) "Near-memory computing: Past, present, and future." Microprocessors and Microsystems 71: 102868.

[3] Mutlu, Onur, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. (2020) "A modern primer on processing in memory." arXiv preprint arXiv:2012.03112.

[4] Devaux, Fabrice. (2019) "The true processing in memory accelerator." In Proceedings of the IEEE Hot Chips 31 Symposium (HCS), Stanford, CA, USA.

[5] Kwon, Young-Cheon, et al. (2021) "25.4 A 20nm 6GB function-in-memory DRAM, based on HBM2 with a 1.2TFLOPS programmable computing unit using bank-level parallelism, for machine learning applications." In Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC): 350–352.

[6] Ghose, Saugata, Amirali Boroumand, Jeremie S. Kim, Juan Gómez-Luna, and Onur Mutlu. (2019) "Processing-in-memory: A workload-driven perspective." IBM Journal of Research and Development 63 (6): 3:1–3:19.

[7] Boroumand, Amirali, et al. (2018) "Google workloads for consumer devices: Mitigating data movement bottlenecks." In Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS): 316–331.

[8] Oliveira, Geraldo F., Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. (2022) "Methodologies, workloads, and tools for processing-in-memory: Enabling the adoption of data-centric architectures." In Proceedings of the IEEE Computer Society Annual Symposium on VLSI (ISVLSI); arXiv preprint arXiv:2205.14647.

[9] Gómez-Luna, Juan, Yuxin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F. Oliveira, Gagandeep Singh, and Onur Mutlu. (2023) "Evaluating machine learning workloads on memory-centric computing systems." In Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), doi:10.1109/ISPASS57527.2023.00013.

[10] Item, Maurus, Juan Gómez-Luna, Yuxin Guo, Geraldo F. Oliveira, Mohammad Sadrosadati, and Onur Mutlu. (2023) "TransPimLib: Efficient transcendental functions for processing-in-memory systems." In Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS): 235–247.

[11] Ke, Liu, et al. (2020) "RecNMP: Accelerating personalized recommendation with near-memory processing." In Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA); arXiv preprint arXiv:1912.12953.

[12] Cooper, Keith D., and Linda Torczon. (2022) Engineering a Compiler, 3rd edition. Cambridge, MA, USA: Morgan Kaufmann.

[13] Gómez-Luna, Juan, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. (2022) "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system." IEEE Access 10: 52565–52608.

[14] Devic, Alexandar, Siddhartha Balakrishna Rai, Anand Sivasubramaniam, Ameen Akel, Sean Eilert, and Justin Eno. (2022) "To PIM or not for emerging general purpose processing in DDR memory systems." In Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA): 231–244.

[15] Khan, Asif Ali, Hamid Farzaneh, Karl F. A. Friebel, Clément Fournier, Lorenzo Chelini, and Jeronimo Castrillon. (2023) "CINM (Cinnamon): A compilation infrastructure for heterogeneous compute in-memory and compute near-memory paradigms." arXiv preprint arXiv:2301.07486.

[16] Chen, Jinfan, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. (2023) "SimplePIM: A software framework for productive and efficient processing-in-memory." In Proceedings of the 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT): 99–111.

[17] Oliveira, Geraldo F., Alain Kohli, David Novo, Juan Gómez-Luna, and Onur Mutlu. (2023) "DaPPA: A data-parallel programming framework for processing-in-memory architectures." arXiv preprint arXiv:2310.10168.

[18] Shin, Yongwon, Juseong Park, Sungjun Cho, and Hyojin Sung. (2023) "PIMFlow: Compiler and runtime support for CNN models on processing-in-memory DRAM." In Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization (CGO): 249–262.

[19] Oliveira, Geraldo F., Juan Gómez-Luna, Lois Orosa, Saugata Ghose, Nandita Vijaykumar, Ivan Fernandez, Mohammad Sadrosadati, and Onur Mutlu. (2021) "DAMOV: A new methodology and benchmark suite for evaluating data movement bottlenecks." IEEE Access 9; arXiv preprint arXiv:2105.03725.

[20] Giannoula, Christina, Ivan Fernandez, Juan Gómez-Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. (2022) "SparseP: Towards efficient sparse matrix vector multiplication on real processing-in-memory architectures." Proceedings of the ACM on Measurement and Analysis of Computing Systems 6 (1): Article 21.

[21] Boroumand, Amirali, Saugata Ghose, Minesh Patel, Hasan Hassan, Brandon Lucia, Rachata Ausavarungnirun, Kevin Hsieh, Nastaran Hajinazar, Krishna T. Malladi, Hongzhong Zheng, and Onur Mutlu. (2019) "CoNDA: Efficient cache coherence support for near-data accelerators." In Proceedings of the 46th International Symposium on Computer Architecture (ISCA): 629–642.

[22] Wang, Zhengrong, Christopher Liu, Aman Arora, Lizy John, and Tony Nowatzki. (2023) "Infinity Stream: Portable and programmer-friendly in-/near-memory fusion." In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3: 359–375.

[23] Oliveira, Geraldo F., Ataberk Olgun, Abdullah Giray Yağlıkçı, F. Nisa Bostancı, Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. (2024) "MIMDRAM: An end-to-end processing-using-DRAM system for high-throughput, energy-efficient and programmer-transparent multiple-instruction multiple-data computing." In Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA): 186–203.

[24] Hajinazar, Nastaran, Geraldo F. Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, and Onur Mutlu. (2021) "SIMDRAM: A framework for bit-serial SIMD processing using DRAM." In Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), doi:10.1145/3445814.3446749.

[25] Lee, Sukhan, et al. (2021) "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product." In Proceedings of the ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA): 43–56.

[26] Hyun, Bongjoon, Taehun Kim, Dongjae Lee, and Minsoo Rhu. (2024) "Pathfinding future PIM architectures by demystifying a commercial PIM technology." In Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA): 263–279.

[27] Sun, Xiaotian, et al. (2023) "PIMCOMP: A universal compilation framework for crossbar-based PIM DNN accelerators." In Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC); arXiv preprint arXiv:2307.01475.

[28] Free Software Foundation. (2021) "GNU Bison: The Yacc-compatible parser generator, version 3.8." Available: https://www.gnu.org/software/bison/manual/.

[29] Free Software Foundation. (2024) "Using the GNU Compiler Collection (GCC)." Available: https://gcc.gnu.org/onlinedocs/.

[30] The Matplotlib Development Team. (2024) "Matplotlib: Visualization with Python." Available: https://matplotlib.org/.